Python语言与系统设计

（函数的设计与使用）

Python Language & System Design

第5章 函数的设计与使用

(Design and Use of Functions)



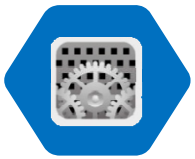
合肥工业大学 计算机与信息学院

第5章 函数的设计与使用


5.1 函数的定义


5.2 形参与实参


5.3 参数类型


5.4 return语句


5.5 Lambda表达式


5.6 高级话题


5.7 本章小结



5.1 函数的定义

- Python中函数的定义使用def保留字，语法形式如下：

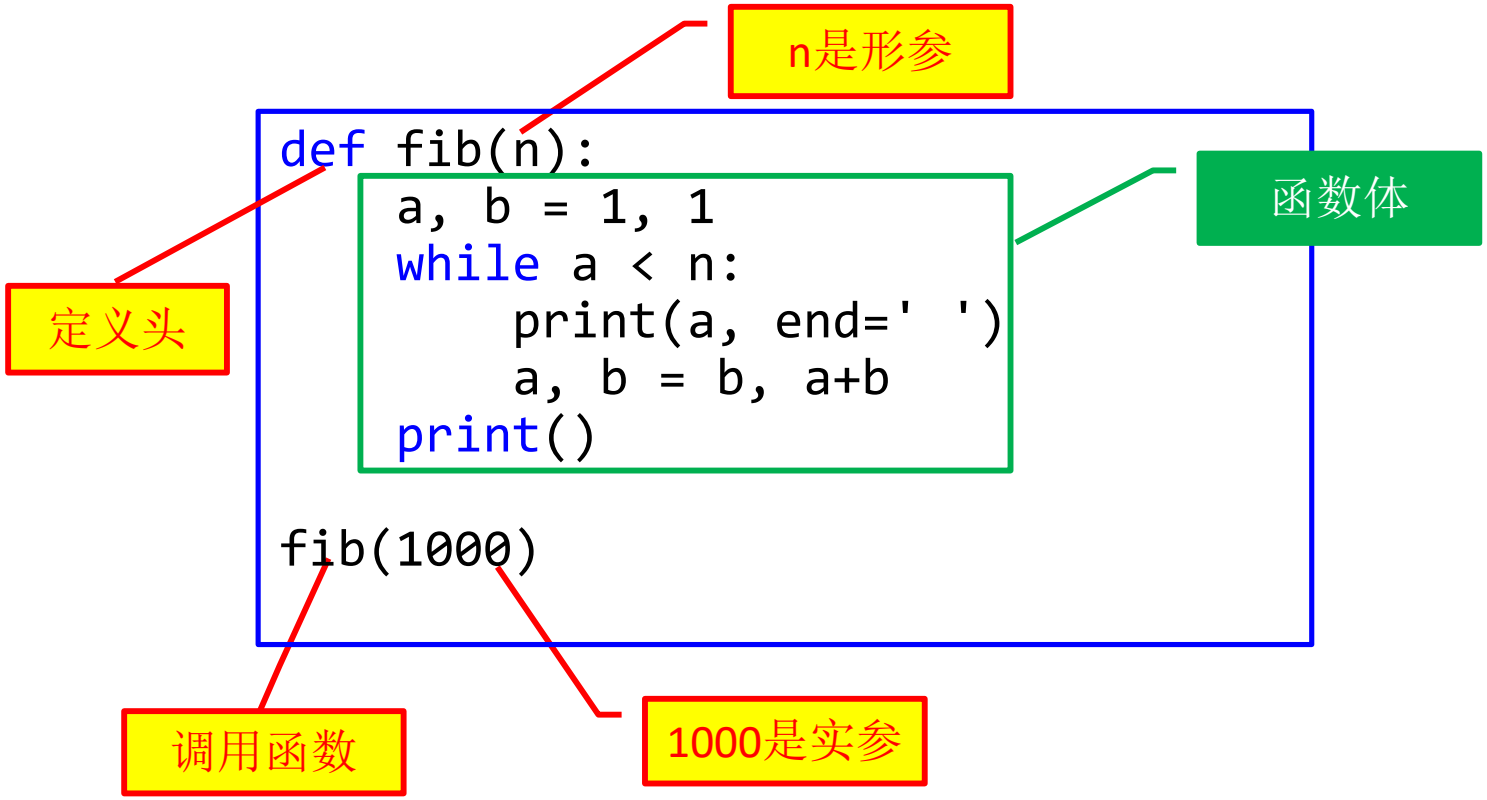
```
def <函数名>(<参数列表>):
    '''注释'''
    <函数体>
    return <返回值列表>
```

- 注意事项
  - ✓ 函数形参不需要声明类型，也不需要指定函数返回值类型
  - ✓ 即使该函数不需要接收任何参数，也必须保留一对空的圆括号
  - ✓ 括号后面的冒号必不可少
  - ✓ 函数体相对于def关键字必须保持一定的空格缩进
  - ✓ Python允许嵌套定义函数

5.1 函数的定义

- 生成斐波那契数列的函数定义和调用

数列：1、1、2、3、5、8、13、21……..





5.1 函数的定义

- Python中的函数和自定义对象的成员也是可以随时发生改变的，可以为函数和自定义对象动态增加新成员。

```
>>> def func():
    print(func.x)
>>> func()
AttributeError: 'function' object has no attribute 'x'
>>> func.x = 3                          #动态为函数增加新成员
>>> func()
3
>>> del func.x                          #删除函数成员
```

5.2 形参与实参

- 函数定义时括弧内为形参。如果有多个形参，需要使用逗号进行分隔。
- 函数调用时向其传递实参 。（实参的值或引用）
- 对于绝大多数情况下，在函数内部直接修改形参的值不会影响实参，而是创建一个新变量。

```
>>> def addOne(a):
            print(a)
            a += 10
            print(a)


>>> a=5
>>> addOne(a)
5
15
>>> a
5
>>>
```

5.2 形参与实参

- 有些情况下，可以通过特殊的方式在函数内部修改实参的值。

```
>>> def modify(v):                    # 使用下标修改列表元素值
    v[0] = v[0]+1
>>> a = [2,30,40,50]
>>> modify(a)
>>> a
[3, 30, 40, 50]
>>> def modify(v, item):              # 使用列表的方法为列表增加元素
    v.append(item)
>>> a = [2]
>>> modify(a,3)
>>> a
[2, 3]
```

- 如果传递给函数的实参是可变序列，并且在函数内部使用下标或可变序列自身的方法增加、删除元素或修改元素时，实参也得到相应的修改。

```
>>> def modify(d):
        d['age'] = 18


>>> a = {'name':'zhang', 'age':26}
>>> modify(a)
>>> a
{'name': 'zhang', 'age': 18}
>>>
```

5.3 参数类型

- Python在定义函数时不需要指定形参的类型，完全由调用者传递的实参类型以及Python解释器的理解和推断来决定，类似于重载和泛型。

- 位置参数（**positional arguments**）是比较常用的形式，调用函数时实参和形参的顺序必须严格一致，并且实参和形参的数量必须相同。

```
>>> def demo(a, b, c):
    print(a, b, c)

>>> demo(3, 4, 5)
3 4 5
>>> demo(1, 2, 3, 4)
TypeError: demo() takes 3 positional arguments but 4 were
given
```

5.3 参数类型

➢ 默认值参数

- 调用带有默认值参数的函数时，可以不对默认值参数进行赋值，也可以为其赋值，具有很大的灵活性。

```
>>> def say( message, times =1 ):
    print(message * times)

>>> say('hello')
hello
>>> say('hello',3)
hello hello hello
```

5.3 参数类型

- 注意：默认值参数必须出现在函数参数列表的最右端，任何一个默认值参数右边不能有非默认值参数。

```
>>> def func(a=3, b, c=5):
    print(a, b, c)
```

错误

```
>>> def func(a=3, b):
     print(a, b)
```

错误

```
>>> def func(a, b, c=5):
    print(a, b, c)
```

5.3 参数类型

- 另外，默认值参数如果使用不当，会导致很难发现的逻辑错误，例如：

```
def demo(newitem,old_list=[]):
    old_list.append(newitem)
    return old_list

print(demo('a'))
print(demo('b'))
print(demo('c'))
```



这段代码会输出什么呢？

```
['a']
['a', 'b' ]
['a', 'b', 'c']
```

- 原因在于默认值参数的赋值只会在函数定义时被解释一次。当使用可变序列作为参数默认值时，一定要谨慎操作。

5.3 参数类型

- 默认值参数
  - 解决方案：

不对形参old_list进行赋值等操作，
所以old_list一直为空列表

```
def demo(newitem,old_list=None):
    if old_list is None:
        old_list=[]
    new_list = old_list[:]
    new_list.append(newitem)
    return new_list

print(demo('a'))
print(demo('b'))
print(demo('c'))
```

```
['a']
['b']
['c']
```

```
def demo(newitem,old_list=[]):
    new_list = old_list[:]
    new_list.append(newitem)
    return new_list

print(demo('a'))
print(demo('b'))
print(demo('c'))
```

5.3 参数类型

关键参数

- 通过关键参数，实参顺序可以和形参顺序不一致，但不影响传递结果，避免了用户需要牢记位置参数顺序的麻烦。

```
>>> def demo(a,b,c=5):
    print(a,b,c)

>>> demo(3,7)
3 7 5
>>> demo(c=8,a=9,b=0)
9 0 8
```

可变长度参数

- 可变长度参数主要有两种形式：在参数名前加1个*或2个**
  - *parameter用来接收多个位置实参并将其放在一个元组中
  - **parameter接收多个关键参数并存放到字典中

5.3 参数类型

➢ 可变长度参数

- *parameter的用法

```
>>> def demo(*p):      #参数放在一个元组中
    print(p)

>>> demo(1,2,3,4,5,6,7)
(1, 2, 3, 4, 5, 6, 7)
```

- **parameter的用法

自动接收的参数转换成字典

```
>>> def demo(**p):
    for item in p.items():
        print(item)

>>> demo(x=1,y=2,z=3)
('y', 2)
('x', 1)
('z', 3)
```

```
>>> def demo(**p):
    for item in p.values():
        print(item)

>>> demo(x=1,y=2,z=3)
1
2
3
```

- 可变长度参数
  - 几种不同类型的参数可以混合使用，但是不建议这样做。

```
>>> def func_4(a, b, c=4, *aa, **bb):
    print(a,b,c)
    print(aa)
    print(bb)

>>> func_4(1,2,3,4,5,6,7,8,9,xx='1',yy='2',zz=3)
1 2 3
(4, 5, 6, 7, 8, 9)
{'yy': '2', 'xx': '1', 'zz': 3}
```

5.3 参数类型

参数传递的序列解包

- 传递参数时，可以通过在实参序列前加一个星号将其解包，然后传递给多个单变量形参。

```
>>> def demo(a, b, c):
    print(a+b+c)

>>> seq = [1, 2, 3]
>>> demo(*seq)
6
>>> tup = (1, 2, 3)
>>> demo(*tup)
6
>>> Set = {1, 2, 3}
>>> demo(*Set)
6
```

```
>>> dic = {1:'a', 2:'b', 3:'c'}
>>> demo(*dic)  #默认使用字典的键
6
>>> demo(*dic.values())#使用字典的值
abc
```

5.4 return语句与变量作用域

return语句

- return语句用来从一个函数中返回一个值，同时结束函数。
- Python将认为该函数以return None结束，返回空值：
  - 函数没有return语句；
  - 函数有return语句但是没有执行到；
  - 函数有return但是没有返回任何值。

- 在调用函数或对象方法时，一定要注意有没有返回值，这决定了该函数或方法的用法。

```
>>> a_list = [1, 2, 3, 4, 9, 5, 7]
>>> print(sorted(a_list))
[1, 2, 3, 4, 5, 7, 9]
>>> print(a_list)
[1, 2, 3, 4, 9, 5, 7]
>>> print(a_list.sort()) #对象方法sort()原地操作，没有返回值
None
>>> print(a_list)
[1, 2, 3, 4, 5, 7, 9]
```

5.4 return语句与变量作用域

变量作用域

- 变量起作用的代码范围称为变量的作用域，不同作用域内变量名可以相同，互不影响。
- 在函数内部定义的普通变量只在函数内部起作用，称为局部变量。当函数执行结束后，局部变量自动删除，不再可以使用。
- 局部变量的引用比全局变量速度快，应优先考虑使用。
- 全局变量可以通过关键字global来定义。这分为两种情况：
  - ✓ 一个变量已在函数外定义，如果在函数内需要为这个变量赋值，并要将这个赋值结果反映到函数外，可以在函数内使用global将其声明为全局变量。
  - ✓ 如果一个变量在函数外没有定义，在函数内部也可以直接将一个变量定义为全局变量，该函数执行后，将增加一个新的全局变量。

5.4 return语句与变量作用域

- 也可以这么理解：
  - ✓ 在函数内只引用某个变量的值而没有为其赋新值，如果这样的操作可以执行，那么该变量为（隐式的）全局变量；
  - ✓ 如果在函数内任意位置有为变量赋新值的操作，该变量即被认为是（隐式的）局部变量，除非在函数内显式地用关键字global进行声明。

```
>>> def demo():
    global x
    x = 3
    y = 4
    print(x,y)

>>> x = 10    #定义全局变量x
>>> demo()
3  4
>>> x  #全局变量被修改
3
>>> y  #局部变量函数运行结束后被删除
NameError: name 'y' is not defined
```

```
>>> del x
>>> x
NameError: name 'x' is not defined
>>> demo()  #函数创建了全局变量x
3  4
>>> x
3
```

5.4 return语句与变量作用域

变量作用域

- 注意：在某个作用域内任意位置只要有为变量赋值的操作，该变量在这个作用域内就是局部变量，除非使用global进行了声明。

```
>>> x = 3
>>> def f():
    print(x)        #本意是先输出全局变量x的值，但是不允许这样做
    x = 5                   #有赋值操作，因此在整个作用域内x都是局部变量
    print(x)

>>> f()
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    f()
  File "<pyshell#9>", line 2, in f
    print(x)
UnboundLocalError: local variable 'x' referenced before assignment
```

```
>>> x = 3

>>> def f():
    print(x)

>>> f()
3
```

变量作用域

- 如果局部变量与全局变量具有相同的名字，那么该局部变量会在自己的作用域内隐藏同名的全局变量。

```
>>> def demo():
    x = 3  #局部变量
    print(x)
>>> x = 10  #全局变量
>>> x
10
>>> demo()
3
>>> x      #函数执行不影响外面全局变量的值
10
```

5.4 return语句与变量作用域

变量作用域

- 如果需要在同一个程序的不同模块之间共享全局变量的话，可以编写一个专门的模块来实现这一目的。例如，假设在模块A.py中有如下变量定义：

```
x =100
```

- 在模块C.py中有以下语句来访问全局变量的值：

```
import A
print(A.x)
```

5.5 Lambda表达式

- lambda表达式可以用来声明匿名函数，又称**lambda函数**
  - 也就是没有函数名字的临时使用的小函数。

- 匿名函数并非没有名字，而是将函数名作为函数结果返回：
  - <函数名> = lambda <参数列表>: <表达式>
- lambda函数与正常函数一样，等价于下面形式：

```
def <函数名>(<参数列表>):
    return <表达式>
```

等价于

```
def  f(x,y,z):
    return x+y+z
```

```
>>> f = lambda x, y, z: x+y+z          #可以给lambda表达式起名字
>>> f(1,2,3)                           #像函数一样调用
6
>>> g = lambda x, y=2, z=3: x+y+z      #参数默认值
>>> g(1)
6
```

5.5 Lambda表达式

✓ 示例

```
>>> L = [(lambda x: x**2),
         (lambda x: x**3),
         (lambda x: x**4)]
>>> print(L[0](2),L[1](2),L[2](2))
4 8 16
>>> L = [1,2,3,4,5]
>>> print(list(map(lambda x: x+10, L)))
[11, 12, 13, 14, 15]
>>> L
[1, 2, 3, 4, 5]
```

```
>>> def demo(n):
    return n*n

>>> demo(5)
25
>>> a_list = [1,2,3,4,5]
>>> list(map(lambda x: demo(x), a_list))  #在lambda表达式中调用函数
[1, 4, 9, 16, 25]
```

5.5 Lambda表达式

- 示例

```
>>> data = list(range(20))
>>> data
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
>>> import random
>>> random.shuffle(data)
>>> data
[6, 17, 3, 7, 11, 18, 12, 1, 19, 15, 4, 8, 9, 13, 5, 10, 14, 16, 2, 0]
>>> data.sort(key=lambda x: x)
>>> data                                          #和不指定规则效果一样
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

>>> data.sort(key=lambda x: len(str(x)))          #按转换成字符串以后的长度排序
>>> data
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
>>> data.sort(key=lambda x: len(str(x)), reverse=True)          #降序排序

>>> data
[10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

5.6 案例应用

✓ 例5-1：编写函数计算圆的面积。

```
from math import pi as PI

def CircleArea(r):
    if isinstance(r, (int,float)) and r>0:
        return PI*r*r
    else:
        print('You must give an integer or float as radius.')

print(CircleArea(3))
```

5.6 案例应用

- ✓ 例5-2：编写函数，接收任意多个实数，返回一个元组，其中第一个元素为所有参数的平均值，其他元素为所有参数中大于平均值的实数。

```
def demo(*para):
    avg = sum(para)/len(para)
    g = [i for i in para if i>avg]
    return (avg,)+tuple(g)

print(demo(1,2,3,4,5,6))
```

```
(3.5, 4, 5, 6)
>>>
```

5.6 案例应用

✓ 例5-3：编写函数，接收字符串参数，返回一个元组，其中第一个元素为大写字母个数，第二个元素为小写字母个数。

```
def demo(s):
    result = [0,0]
    for ch in s:
        if 'a'<=ch<='z':
            result[1] += 1
        elif 'A'<=ch<='Z':
            result[0] += 1
    return tuple(result)

print(demo('aaaabbbbC'))
```

5.6 案例应用

✓ 例5-4：编写函数，接收包含20个整数的列表lst和一个整数k作为参数，返回新列表。处理规则为：将列表lst中下标k之前的元素逆序，下标k之后的元素逆序，然后将整个列表lst中的所有元素再逆序。

```
def demo(lst,k):
    x = lst[:k]
    x.reverse()
    y = lst[k:]
    y.reverse()
    r = x+y
    r.reverse()
    return r

lst = list(range(1,21))
print(demo(lst,5))
```

```
[6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 1, 2, 3, 4, 5]
```

5.6 案例应用

✓ 例5-5：编写函数，接收整数参数t，返回斐波那契数列中大于t的第一个数。

```
def demo(t):
    a, b = 1, 1
    while b<t:
        a, b = b, a+b
    else:
        return b

print(demo(50))
```

或者：

```
def demo(t):
    a, b = 1, 1
    while b<t:
        a, b = b, a+b
    return b
```

5.6 案例应用

✓ 例5-6：编写函数，接收一个包含若干整数的列表参数lst，返回一个元组，其中第一个元素为列表lst中的最小值，其余元素为最小值在列表lst中的下标。

```
import random
def demo(lst):
    m = min(lst)
    result = (m,)
    for index, value in enumerate(lst):
        if value==m:
            result = result+(index,)
    return result

x = [random.randint(1,20) for i in range(50)]
print(x)
print(demo(x))
```

```
[18, 7, 10, 13, 16, 20, 20, 18, 14, 12, 2, 5, 6, 13, 7, 15, 15,
 9, 8, 19, 3, 9, 7, 9, 6, 13, 17, 15, 7, 5, 19, 13, 18, 13, 17,
 14, 1, 6, 10, 4, 4, 12, 10, 10, 4, 6, 10, 1, 18, 18]
>>> print(demo(x))

(1, 36, 47)
```

5.6 案例应用

✓ 例5-7：编写函数，接收一个正偶数为参数，输出两个素数，并且这两个素数之和等于原来的正偶数。如果存在多组符合条件的素数，则全部输出。

```
import math

def IsPrime(n):
    m = int(math.sqrt(n))+1
    for i in range(2, m):
        if n%i==0:
            return False
    return True

def demo(n):
    if isinstance(n,int) and n>0 and n%2==0:
        for i in range(3, n//2 +1):
            if IsPrime(i) and IsPrime(n-i):
                print(i, '+', n-i, '=', n)

demo(60)
```

```
7 + 53 = 60
13 + 47 = 60
17 + 43 = 60
19 + 41 = 60
23 + 37 = 60
29 + 31 = 60
>>>
```

5.6 案例应用

✓ 例5-8：编写函数，接收两个正整数作为参数，返回一个元组，其中第一个元素为最大公约数，第二个元素为最小公倍数。

```
def demo(m,n):
    if m>n:
        m, n = n, m
    p = m*n
    while m!=0:
        r = n%m
        n = m
        m = r
    return (n, p//n)

print(demo(12, 30))
```

- Python标准库已经提供了计算最大公约数的方法。

```
>>> import fractions
>>> fractions.gcd(36, 39)
3
>>> fractions.gcd(30, 20)
10
>>> 30*20/fractions.gcd(30, 20)
60.0
>>> import math
>>> math.gcd(36, 39)
3
```

5.6 案例应用

✓ 例5-9：编写函数，接收一个所有元素值都不相等的整数列表x和一个整数n，要求将值为n的元素作为支点，将列表中所有值小于n的元素全部放到n的前面，所有值大于n的元素放到n的后面。

```
import random
def demo(x, n):
    if n not in x:
        print(n, ' is not an element of ', x)
        return
    i = x.index(n)              #获取指定元素在列表中的索引
    x[0], x[i] = x[i], x[0] #将指定元素与第0个元素交换
    key = x[0]
    i = 0
    j = len(x) - 1
```

```
while i<j:
        while i<j and x[j]>=key: #从后向前寻找第一个比指定元素小的元素
            j -= 1
        x[i] = x[j]
        while i<j and x[i]<=key: #从前向后寻找第一个比指定元素大的元素
            i += 1
        x[j] = x[i]
        x[i] = key

x =list(range(1, 10))
random.shuffle(x)
print(x)
demo(x, 4)
print(x)
```

```
[5, 2, 8, 4, 6, 7, 9, 1, 3]
[3, 2, 1, 4, 6, 7, 9, 5, 8]
>>>
```

- 上面的代码演示的是快速排序算法中非常重要的一个步骤，当然也可以使用下面更加简洁（但效率略差）的代码来实现。

```
>>> import random
>>> def demo(x, n):
    t1 = [i for i in x if i<n] #找出所有小于n的
    t2 = [i for i in x if i>n]
    return t1+[n]+t2

>>> x = list(range(1,10))
>>> random.shuffle(x)
>>> x
[1, 9, 3, 6, 5, 2, 4, 7, 8]
>>> demo(x, 4)
[1, 3, 2, 4, 9, 6, 5, 7, 8]
```

内置函数Map

- 内置函数**map()**可以将一个函数作用到一个序列或迭代器对象上。

```
>>> list(map(str,range(5)))
['0', '1', '2', '3', '4']
>>> def add5(v):
    return v+5

>>> list(map(add5,range(10)))
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]

>>> def add(x, y):return x+y
>>> list(map(add, range(5), range(5)))
[0, 2, 4, 6, 8]
```



5.6 高级话题

➢ 内置函数filter

- 内置函数filter将一个函数作用到一个序列上，返回该序列中使得该函数返回值为True的那些元素组成的filter对象。

```
>>> seq=['foo','x41','?!','***']
>>> def func(x):
    return x.isalnum()
>>> list(filter(func,seq))
['foo', 'x41']
>>> seq
['foo', 'x41', '?!', '***']
>>> [x for x in seq if x.isalnum()]
['foo', 'x41']
>>> list(filter(lambda x:x.isalnum(),seq))
['foo', 'x41']
```

reduce函数

- 标准库**functools**中的**reduce()**函数可以将一个接受**2**个参数的函数以迭代的方式从左到右依次作用到一个序列或迭代器对象的所有元素上。

```
>>> from functools import reduce
>>> seq=[1,2,3,4,5,6,7,8,9]
>>> reduce(lambda x,y:x+y, seq)
45
>>> def add(x, y):
    return x + y
>>> reduce(add,range(10))
45
>>> reduce(add,map(str,range(10)))
'0123456789'
```

➢ 生成器函数

- 包含**yield**语句的函数可以用来创建生成器对象，这样的函数也称生成器函数。
- 每次执行到yield语句会返回一个值然后暂停或挂起后面代码的执行，下次通过生成器对象的__next__()方法、内置函数next()、for循环遍历生成器对象元素或其他方式显式“索要”数据时恢复执行。

```
def fab(max):
    n, a, b = 0, 0, 1
    L = []
    while n < max:
        L.append(b)
        a, b = b, a + b
        n = n + 1
    return L
for n in fab(5):
    print(n)
```

```
-----
1
1
2
3
5
>>>
```

函数在运行中占用的内存会随着参数 max 的增大而增大，如果要控制内存占用，最好不要用 List

```
def fab(max):
    n, a, b = 0, 0, 1
    while n < max:
        yield b       # 使用 yield
        a, b = b, a + b
        n = n + 1

for n in fab(5):
    print(n)
```



5.6 高级话题

```
>>> def f():
    a, b = 1, 1                 #序列解包，同时为多个元素赋值
    while True:
        yield a                 #暂停执行，需要时再产生一个新元素
        a, b = b, a+b           #序列解包，继续生成新元素
```

执行到 yield a 时，f 函数就返回一个迭代值，下次迭代时，代码从 yield a 的下一条语句继续执行，

```
>>> a = f()                     #创建生成器对象
>>> for i in range(10):         #斐波那契数列中前10个元素
    print(a.__next__(), end=' ')
```

```
1 1 2 3 5 8 13 21 34 55
```



5.6 高级话题

- 函数嵌套定义
  - 在Python中，函数是可以嵌套定义的。

```
def myMap(iterable, op, value):          #自定义函数
    if op not in '+-*/':
        return 'Error operator'
    def nested(item):                          #嵌套定义函数
        return eval(repr(item)+op+repr(value))
                                    #repr函数返回一个对象的 string 格式。
    return map(nested, iterable)               #使用在函数内部定义的函数

>>> list(myMap(range(5), '+', 5))              #调用外部函数
[5, 6, 7, 8, 9]
>>> list(myMap(range(5), '-', 5))
[-5, -4, -3, -2, -1]
>>> list(myMap(range(5), '*', 5))
[0, 5, 10, 15, 20]
>>> list(myMap(range(5), '/', 5))
[0.0, 0.2, 0.4, 0.6, 0.8]
```

- 函数嵌套定义

```
def func(a, b):
    print("外层函数调用")
    def line(x):
        print("内层函数调用： a=%d\tx=%d" % (a, x))
        return a * x - b
    return line

line = func(2, 3)  #调用外层函数返回的是内层函数的引用
print(line(5))
```

```
外层函数调用
内层函数调用： a=2        x=5
7
```

5.7 本章小结

内容回顾

- 函数的相关概念
- 形参与实参、参数类型、参数作用
- **函数的其他应用**

思考



- 函数的应用